EECS 498 · AASE · FALL 2026

How LLMs Code (Under the Hood)

LECTURE 02 · SEP 3, 2026

The developer who has internalized the mechanics has an **enormous** advantage.

FOUR IDEAS

Today’s mechanics

Tokenization

How the model sees your code.

Context window

The only resource that matters.

Hallucination

Why the model makes things up.

Capability vs Knowledge

Two axes of model improvement.

Tokenization

WHAT THE MODEL SEES

BPE INTUITION

Tokens, not words

- The model does not see characters
- The model does not see words
- The model sees **tokens** — chunks it has seen in training

Byte-pair encoding: merge most common pairs, repeat until ~100K vocab.

TWO CONSEQUENCES

What tokenization costs you

Unit of cost

Verbose code costs more — in dollars on a vendor, in RAM and seconds on your own machine.

Unit of attention

Every token competes for focus. Your identifiers fragment into common pieces, so they carry no special weight.

9 tokens

```python
def f(x):
    return x * 2
```

~35 tokens

```python
def calculate_double_of_input_value(
    input_value: int
) -> int:
    """Return value times two."""
    return input_value * 2
```

Context Window

THE ONLY RESOURCE THAT MATTERS

What the model sees

- **Inside the window**: files, history, repo map, system prompt
- **Outside the window**: *nothing*
- No DB. No web fetch. No retrieval

This is the single most important fact about LLMs for an engineer.

Three costs of large context

Cost

Tokens billed, or RAM and seconds on your own box. Attention is quadratic: 2× context ≈ 4× work.

Latency

Tens of seconds to first token on a laptop where a tight prompt starts at once.

Quality

Needle-in-haystack: bigger can be *worse*.

PEDAGOGY OF /ADD

Aider's friction is the lesson

```
/add main.py        → file enters context
/drop main.py       → file leaves context
/tokens             → see exactly what's in the window
/clear              → reset history (keep files)
```

Not a limitation. The discipline you'd want anyway.

Context drift

- History accumulates: every turn you sent, every turn it returned
- Fifty messages in, most of the window is *dead weight*
- The model can’t separate what’s true now from what was true at message 3

`/clear` between tasks. Claude Code (L05) compacts for you, and compaction is lossy.

CHAT BREAK · 5 MIN

Talk to your neighbor: when a long chat degrades, what is happening inside the window?

Hallucination

THE MISLEADING WORD

The model is doing what it was trained to do

- Autoregressive sampling
- Pick most likely next token from a distribution
- No fact-check step
- No DB lookup
- No "wait, does this exist?"

Statistical priors win when context is silent

- Training data: 100× more `import requests` than `import httpx`
- Your project uses httpx
- Prompt doesn't mention it
- Model produces `requests` — plausible, wrong for *your* context

THREE FLAVORS IN CODE

Common hallucinations

Invented APIs

user.get_profile() — plausible, not real.

Stale knowledge

pandas 1.x APIs in a 2.0 codebase.

Wrong signatures

Wrong kwarg, wrong dtype — subtle.

When the model needs a fact, **show it the fact**.

THE LIMIT

What context cannot fix

- "What's the latest version of our internal library?"
- The answer isn't in your repo either, so no `/add` produces it
- The model guesses, ***plausibly***, and you cannot tell from the output

Hand it the fact, or build a tool that looks the fact up. You build those in week 5.

Capability vs Knowledge

TWO AXES

Capability

Reasoning skill. Multi-step, cross-file, holding constraints.

Scales with: model size.

qwen3.5:4b → 9b → frontier.

Knowledge

What the model read. Recency. Niche depth.

Scales with: training data size + freshness.

Small + recent can beat large + old.

Why not the top rung?

④

qwen3.5:4b

~2.5 GB. Fails fast and fails obviously. Where you start.

⑨

qwen3.5:9b

~6 GB at Q4. The course default. Part 2 of aider-practice needs it.

Frontier

Someone else's datacenter. Strong on both axes, and you cannot learn on it.

A frontier model covers for you. A 4B ***doesn't***. Two weeks from now you'll have run both.

CHOOSING

Match model to task

- **Cost-sensitive batch** (1000× repeats) → smallest model that clears the bar
- **Reasoning-heavy single task** → the most capable model you have
- **Niche library or framework** → capability won’t save you. That’s a *context* problem
- **Anything you’re learning on** → the rung that fails where you erred

Usually right in production, usually wrong in a classroom. The skill is telling which room you're in.

Diagnostic

WHEN OUTPUT IS WRONG

ASK IN ORDER

Three questions

1. **Context** — did the model have what it needed?
2. **Hallucination** — did it invent an API?
3. **Capability or knowledge** — task too hard? too new?

Next lecture: this becomes the Big Three. You'll use it forever.

Tools wrap the model. The **core inference call** has no lookup.

What to take away

- **Tokens** are the unit of cost and the unit of attention
- **The context window** is the only resource that matters. Spend it deliberately
- **Hallucination** is pattern completion with no lookup. Fix it with context
- **Capability and knowledge** are different axes. Big against recent

L03 stacks the Big Three (Context, Model, Prompt) on top of these four.

THIS WEEK

`aider-practice` Part 1

- Setup gate and all sixteen lessons due **Fri Sep 11**
- Re-read your own Aider chat logs with today's vocabulary
- Every mechanic from this lecture is *already in them*

Next: L03, the Big Three. Tuesday.

Questions?